

A dynamic program for cross-dating tree rings

PAUL C. VAN DEUSEN

*Institute for Quantitative Studies, USDA Forest Service, Southern Forest Experiment Station,
701 Loyola Avenue, New Orleans, LA 70113, U.S.A.*

Received February 3, 1989

Accepted August 30, 1989

VAN DEUSEN, P. C. 1990. A dynamic program for cross-dating tree rings. *Can. J. For. Res.* **20**: 200–205.

An algorithm for cross-dating tree rings based on dynamic programming is presented. The algorithm considers all possible combinations of missing and double rings while minimizing a squared error loss function in relation to a reference chronology. There is a provision for the user to increase the penalty associated with inserting missing or double rings. Therefore, the algorithm is not totally objective and depends on informed user interaction and repeated application to perform successfully. The end result is a list of years where the algorithm suggests that missing or double rings have occurred.

VAN DEUSEN, P. C. 1990. A dynamic program for cross-dating tree rings. *Can. J. For. Res.* **20** : 200–205.

Nous présentons ici un algorithme reposant sur un mode de programmation dynamique pour effectuer l'interdatation de cernes annuels de croissance des arbres. L'algorithme prend en compte toutes les combinaisons possibles de cernes manquants et de faux cernes en minimisant le carré des erreurs d'une fonction de perte par rapport à une chronologie de référence. L'utilisateur peut accroître à volonté la pénalité due à l'insertion de cernes manquants ou de faux cernes. L'utilisation de l'algorithme n'est donc pas entièrement objective et sa performance repose sur l'expérience de l'utilisateur et sur une utilisation fréquente. Il en ressort finalement une liste d'années pour lesquelles l'algorithme propose l'existence de cernes manquants ou de faux cernes.

[Traduit par la revue]

Introduction

Tree rings have varied uses ranging from studying the effects of pollution on forests to dating archaeological sites. The common thread that ties these different uses is cross dating. Each ring in each tree-ring series must be associated with its exact year of formation, or any subsequent analysis will be of little value. Cross dating is made possible by the fact that trees under similar climatic conditions show similar high-frequency growth patterns. For example, Fig. 1 shows two red oak (*Quercus rubra*) cores from the same stand in Pennsylvania. In these cores, both the low-frequency (long-term) and high-frequency (year to year) trends are quite similar. Cross dating is usually performed by first transforming the data in some way to remove long-term trends and then matching the series of interest with an average of other already cross-dated series. Figure 2 shows the same cores as in Fig. 1 after transforming by taking first differences of natural logarithms. This enhances the year to year variations and makes the variance more homogeneous over time. In fact, the raw data (Fig. 1) might be better for a purely visual approach to cross dating, but standardized data are usually preferred for computer-aided methods.

Currently available computer-aided methods operate by comparing a subject core with an average chronology of other cores that are believed to be correctly cross-dated. The programs then proceed to "slide" the subject core back and forth to find the location where the correlation with the known chronology is greatest. Baillie and Pilcher (1973) presented a computer program that performs this sliding process and gives resulting correlation coefficients and their associated *t*-values. Holmes (1983) developed a more sophisticated program that aids in locating problem areas within the core by breaking it into sections and sliding individual sections. He suggests that segments less than 50 years in length do not provide enough degrees of freedom for this method to be useful. Munro (1984) and Yamaguchi (1986) discuss ways

to transform the data to meet more fully the assumptions of correlation analysis.

If each tree core were complete, then much of the difficulty of cross dating would be eliminated. However, unusual climatic conditions can cause trees to form a double ring in a single year or an entire year's ring to be missing. Thus, simply counting in from the bark will not lead to correctly cross-dated material (Fritts 1976). Currently available procedures can locate cores that are incorrectly cross-dated, but they are unable to locate the exact problem years. The purpose of this paper is to describe a dynamic programming algorithm that attempts to locate the specific years where missing or double rings occurred.

Algorithm to compare a subject core with a chronology

In most dendroecological work, cores are taken from living trees and cross dating and measuring are the first steps after collecting. The date of the outermost ring is assumed to be known in this case. To begin the description of the proposed algorithm, consider Fig. 3, which schematically depicts the process of cross-dating a single core. The intersections of horizontal and vertical lines in Fig. 3 will be called nodes, and the algorithm will visit all nodes in the diagram that are on or within the boundary lines in Fig. 3. The usual dynamic programming terminology (Dreyfus and Law 1977) would refer to the horizontal steps (years) as stages and the vertical steps as states. The proposed algorithm takes one subject core at a time and evaluates the following three decisions that are schematically depicted in Fig. 3: (i) insert a missing ring (depicted as going up in Fig. 3), (ii) accept the ring as corresponding to the current year (go across in Fig. 3), and (iii) assume a double ring and add year *t* to year *t* - 1 (go down in Fig. 3). Note that other decisions could be added and also that the algorithm runs backward in time, so year 1 corresponds to the most recently formed complete ring at the time of coring. In this notation, year *t* - 1 is actually a year ahead of year *t*, but

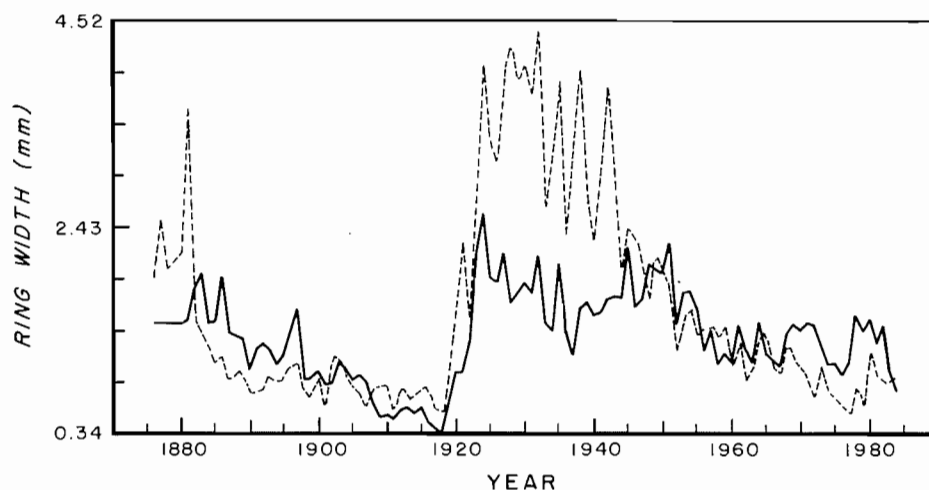


FIG. 1. Raw data from two red oak cores from the same stand in Pennsylvania.

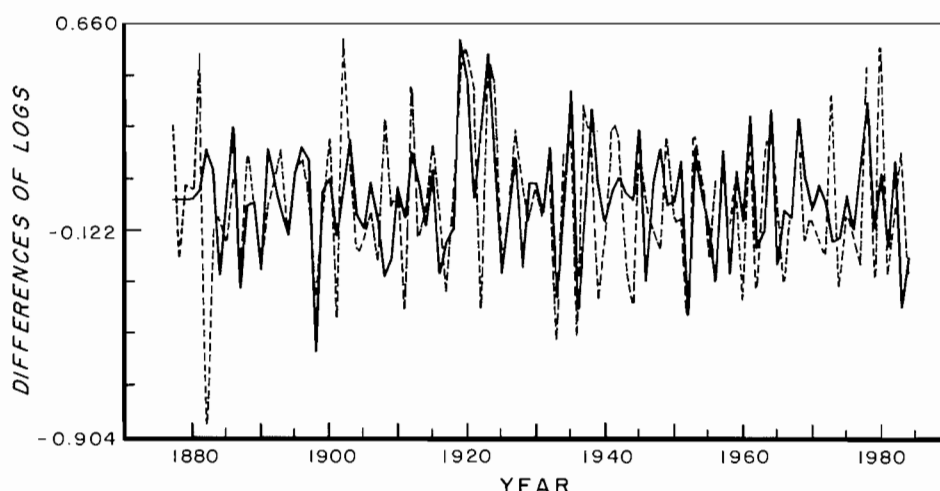
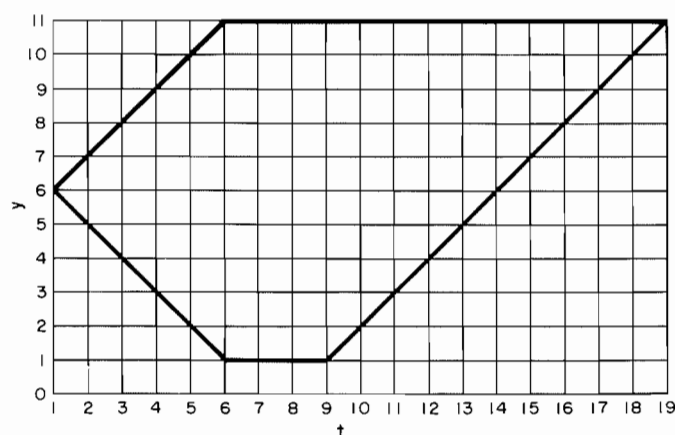


FIG. 2. The same two cores shown in Fig. 1 after transforming by taking first differences of the inverse-hypersine transformation.

the notation used here is very easy to translate into computer code. Also, because the algorithm evaluates all three decisions at all years including year 1, even the outermost ring can be assumed to be double or missing.

The boundary lines represent the limit of acceptable solutions to the problem and are specified by the number of missing or double rings allowed sequentially; the user determines this number, which also determines the number of nodes that the algorithm has to evaluate. In the notation used here, the number mid is 1 more than the number of allowed sequential "missings" or "doubles"; Fig. 3 depicts a problem where five are allowed. The right-hand boundary is determined by the length of the subject core, which is 14 in Fig. 3. Thus, the do-nothing or middle path ends at 14; paths where one more missing than double is inserted finish at 15, and so forth.

A useful conceptual way to view this algorithm is to think of Fig. 3 as representing all possible cross-dating paths. The path that goes straight across represents the do-nothing path, or accepting the subject core as already correctly cross-dated. A path with one up means one missing ring was inserted, and a path with one down means one double ring occurred and so forth. The proposed algorithm simply evaluates all possible paths in the most efficient way possible by using

FIG. 3. A schematic representation of the proposed dynamic programming algorithm. The thicker lines within the rectangular grid represent the boundaries of the problem. t , time in years; y , state variable.

dynamic programming. At each node of Fig. 3, the three decisions are evaluated and a cost is incurred. This cost is added to the costs incurred previously, and we proceed until all nodes and time periods have been visited. At the end of

the core, the least-cost path is deemed to represent the correctly cross-dated core. Subjectivity enters this algorithm in deciding how to transform the data and how to assess the costs of different paths.

Notation and specific formulation

Before addressing the issue of assigning costs to the various paths depicted in Fig. 3, the notation will be defined. As mentioned earlier, the term mid is one more than the number of missing or double rings allowed sequentially. The remaining notation is subscripted to refer to a specific node in Fig. 3:

$$\begin{aligned} S[y, t] &= \text{minimum cost to go from node (mid, 1) to } (y, t) \\ a_u[y, t] &= \text{the cost to go from node } (y-1, t-1) \text{ to } (y, t) \\ a_m[y, t] &= \text{the cost to go from node } (y, t-1) \text{ to } (y, t) \\ a_d[y, t] &= \text{the cost to go from node } (y+1, t-1) \text{ to } (y, t) \end{aligned}$$

In terms of the above notation, the dynamic programming algorithm to solve the cross-dating problem as posed here is

$$S[y, t] = \min \begin{cases} a_u[y, t] + S[y-1, t-1] \\ a_m[y, t] + S[y, t-1] \\ a_d[y, t] + S[y+1, t-1] \end{cases}$$

To initialize the algorithm, $S[\text{mid}, 0] = 0$, and the boundary lines in Fig. 3 are defined as follows:

$$\begin{aligned} \text{upper bound}^1 &= [y, t: y = \text{mid} + \min(t-1, \text{mid}-1)] \\ \text{lower bound} &= [y, t: y = \max(\text{mid} - (t-1), 1)] \\ \text{right bound} &= [y, t: t = n\text{ring} + (y - \text{mid})] \end{aligned}$$

where $n\text{ring}$ equals the number of rings in the subject core.

The answer or the best path is defined to be the path with the minimum cost, which is determined by comparing the S -values associated with the nodes along the right boundary. Staying on and within the boundary implies that certain decisions are not allowed, e.g., you cannot go down when $y = 1$, and you cannot go up when $y = 2(\text{mid}) - 1$.

Cost definitions

Cost will be defined in terms of the squared distance of the subject core from the known chronology. The known chronology value at time t will be called m_t , and the ring width in the j th position in from the bark will be denoted as $r(j)$. It is intentional that no reference is made to $r(t)$ at this point to avoid the implication that the ring width is associated with year t ; this association should only be made after cross dating. The following ring widths from the subject core will be needed in the cost definitions:

$$\begin{aligned} Q_0 &= r(t + \text{mid} - y - 3) \\ Q_1 &= r(t + \text{mid} - y - 2) \\ Q_2 &= r(t + \text{mid} - y - 1) \\ Q_3 &= r(t + \text{mid} - y) \\ Q_{\text{AVE}} &= [Q_3 + r(t + \text{mid} - y + 1)]/2 \end{aligned}$$

¹This statement should be read as follows: the set of nodes at location (y, t) such that $y = \text{mid} + \min(t-1, \text{mid}-1)$, where $\min(\)$ means take the minimum of the arguments.

The Q -values are not subscripted for brevity, but it should be understood that each combination of (y, t) has a different set of Q -values. Notice that if $y = \text{mid}$, then Q_3 equals the ring width in position t , Q_2 is the ring width in position $t-1$, and so forth for Q_1 and Q_0 . If $y > \text{mid}$, then more missings than doubles have been used, so the Q -values lag behind t ; and if $y < \text{mid}$, some Q -values lag ahead of t . In other words, the Q -values contain the appropriate subject core ring widths to associate with each node in the network depicted in Fig. 3. For example, the path that begins with inserting five missing rings uses the ring width in position 1 of the subject core as the $t = 6$ value, whereas the path that begins with five double rings has already used 10 of the subject core ring widths before $t = 6$.

An important feature of the network depicted in Fig. 3 is that regardless of how you arrive at node (y, t) , the same number of subject core ring widths will have been used up. This is because each up move in the network uses no rings, and each down move compensates by using double rings. In fact, the property of being able to disregard nearly all previous decisions in the path leads to efficient solutions via dynamic programming. This allows the whole problem to be solved by solving a series of simple subproblems as one moves through the network; this is referred to as the principle of optimality (Dreyfus and Law 1977).

No transform cost

Costs will be defined for three different cases: (i) the no-transformation case, (ii) the inverse-hypersine case, and (iii) the first-difference case. The no-transformation case occurs when the ring width data are used directly. Of course, this does not rule out the possibility that the data were transformed in advance. The costs for this case are

$$\begin{aligned} a_u[y, t] &= (Q_{\text{AVE}} - m_t)^2 \\ a_m[y, t] &= (Q_3 - m_t)^2 \\ a_d[y, t] &= (Q_2 + Q_3 - m_t)^2 \end{aligned}$$

Although a missing ring is usually designated by a 0, Q_{AVE} is used in defining the cost associated with going up to node (y, t) . In practice, using the average of the nearest ring widths was found to produce better results because the squared cost when using a 0 was often extremely large. This modification brings the cost of inserting a missing ring into a similar scale as the cost of other decisions.

Inverse-hypersine costs

The natural log transformation is often used to make the variance of ring widths from different times and trees more homogeneous. Because 0 is a legitimate ring width value and the log of 0 equals minus infinity, this transformation occasionally causes problems. One solution is to add a small constant such as 1 to each value. Another solution, which has not been used previously with ring widths, is to use the following inverse-hypersine transformation:

$$h(j) = \ln\{r(j) + \sqrt{[r(j)]^2 + 1}\}$$

This transformation equals 0 when $r(j) = 0$, and for larger values of $r(j)$, it gives nearly the same result as simply taking the natural logarithm apart from an additive constant. First differences of inverse-hypersines are nearly identical with first differences of natural logarithms. Therefore, the costs in this section are defined in terms of the inverse hypersine transformation, $h(j)$:

$$a_u[y, t] = [h(Q_{AVE}) - m_t]^2$$

$$a_m[y, t] = (h(Q_3) - m_t)^2$$

$$a_d[y, t] = (h(Q_2 + Q_3) - m_t)^2$$

First-difference costs

When using differences of the data, there are additional restrictions and considerations because differences involve previous decisions. This requires the definition of a new variable to store the decision that was made to arrive at node (y, t) via the cheapest path. Define

$$z[y, t] = 1 \text{ if node } (y, t) \text{ was arrived at by going up}$$

$$z[y, t] = 0 \text{ if } (y, t) \text{ was arrived at by going across}$$

$$z[y, t] = -1 \text{ if } (y, t) \text{ was arrived at by going down}$$

This third variable is essential not only for defining the difference-related costs but also for reconstructing the minimum cost path when the cross-dating algorithm is finished. There are three sets of costs for the first-difference case, which correspond to the three ways of arriving at the node before (y, t) . Because first differences are involved, the decision (way of arriving) at the previous node must also be considered. For example, the decision to go up (insert a missing ring) can only be assigned a proper cost by considering the decision that led to arrival at node $(y - 1, t - 1)$, because this determines the ring width to be subtracted from the value corresponding to the missing ring decision.

The cost definitions for the first-difference case are

$$\text{If } z[y - 1, t - 1] = 1, \quad a_u[y, t] = (0 - m_t)^2$$

$$\text{If } z[y - 1, t - 1] = 0, \quad a_u[y, t] = (Q_{AVE} - Q_3 - m_t)^2$$

$$\text{If } z[y - 1, t - 1] = -1, \quad a_u[y, t] = (Q_{AVE} - Q_2 - Q_3 - m_t)^2$$

$$\text{If } z[y, t - 1] = 1, \quad a_m[y, t] = (Q_3 - Q_{AVE} - m_t)^2$$

$$\text{If } z[y, t - 1] = 0, \quad a_m[y, t] = (Q_3 - Q_2 - m_t)^2$$

$$\text{If } z[y, t - 1] = -1, \quad a_m[y, t] = (Q_3 - Q_2 - Q_1 - m_t)^2$$

$$\text{If } z[y + 1, t - 1] = 1, \quad a_d[y, t] = (Q_3 + Q_2 - Q_{AVE} - m_t)^2$$

$$\text{If } z[y + 1, t - 1] = 0, \quad a_d[y, t] = (Q_3 + Q_2 - Q_1 - m_t)^2$$

$$\text{If } z[y + 1, t - 1] = -1, \quad a_d[y, t] = (Q_3 + Q_2 - Q_1 - Q_0 - m_t)^2$$

The first three costs are the case where a missing ring is to be inserted; this corresponds to coming up to node (y, t) , and the decision that led to arrival at $(y - 1, t - 1)$ determines the appropriate cost. If the previous decision was to insert a missing $z[y - 1, t - 1] = 1$, then that means the previous ring width is a 0. Since the current ring width is also a 0, the cost is $(0 - 0 - m_t)^2$. Note that the known chronology should also be in terms of first differences. If the previous decision were to accept to ring width $z[y - 1, t - 1] = 0$, then Q_{AVE} is used as the missing value and the previous value (Q_3) is subtracted to form the first difference; the cost is therefore $(Q_{AVE} - Q_3 - m_t)^2$.

Similar reasoning is used in defining the remaining costs. For example, the final cost corresponds to arriving at (y, t) by going down (inserting a double ring) when node $(y + 1, t - 1)$ was also arrived at by going down. Thus, the current ring width should be $Q_3 + Q_2$ and the previ-

A

	YEAR						
	7	6	5	4	3	2	1
SUBJECT CORE	M	3	1	4	2	3	0
CHRONOLOGY	3	1	4	2	3	3	0

M = MISSING

B

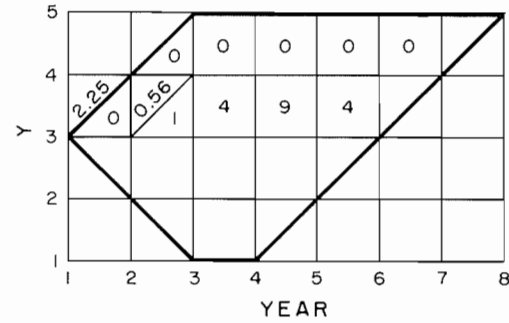


FIG. 4. The schematic representation (B) of applying the cross-dating algorithm to the data given in Fig. 4. The costs for the no-transformation case for several of the possible paths are computed.

ous ring width is $Q_1 + Q_0$, and these form the first difference in the cost function.

Cost functions for first differences of inverse hypersines can be defined by marrying the costs for the inverse-hypersine transformation with the costs for first differences. The most subjective part of these cost definitions involves inserting Q_{AVE} for the value of a missing ring. In practice, further manipulation of the costs associated with missing and double rings may be desirable.

Example application

Figure 4 shows the results of a contrived example application of the method. The manufactured data are depicted for the subject core and a chronology, and only two missings or doubles are allowed sequentially. Notice that a 0 is appended onto both the subject core and the chronology for year 1, which makes year 2 the outermost ring. Appending 0 is a simple method to initialize the algorithm in such a way that all three decisions can be considered for the first actual ring width. In this way the algorithm simply accepts the first 0 ring width as being correctly dated without incurring a penalty; it then can process the remaining ring widths.

Figure 4 shows the costs that would be incurred for the no-transformation case for three of the possible paths. For example, the do-nothing path across the middle costs a total of 18, the path corresponding to inserting a missing value at year 3 and then doing nothing costs 0.5625, and the path corresponding to inserting a missing at year 2 and then doing nothing costs 2.25. The dynamic programming algorithm would decide that the best way to get to node $(4, 3)$ is to accept year 2 and then to insert a missing ring at year 3. By the principal of optimality, the other alternative path gets no further consideration, i.e., inserting a missing ring at

year 2 and then accepting. This is how dynamic programming derives its efficiency; suboptimal paths are dropped the moment they are detected.

For the example depicted in Fig. 4, the path costing only 0.5625 would be deemed correct. Obviously, it might be the case that the path costing 2.25 is the true correct path; the algorithm merely does the job of finding the least cost path based on the way the costs were assigned. Experimentation with actual data where missing and double rings are purposefully injected suggests that the algorithm is quite effective in correctly cross-dating the subject core. However, a missing or a double ring is often inserted from 1 to 3 years away from its true location, and the contrived example gives some insight as to why this occurs. In fact, even a human looking at the original data could make similar errors.

Statistical aids for assessing the results

A computer program has been written (available from author on request) to implement these procedures, and some simple summary statistics are included to help assess the results. The first statistic is based on the ratio of the cost of doing nothing, $COST(nothing)$, i.e., accepting the subject core as is, versus the cost of the cheapest path, $COST(path1)$. Call this statistic $RATIO_{DN}$:

$$RATIO_{DN} = 100[COST(nothing) - COST(path1)]/COST(nothing)$$

$RATIO_{DN}$ gives some information on the relative improvement that the best path made over doing nothing. If this number is small, then the algorithm's result should not be taken too seriously. Whether $RATIO_{DN}$ is large or small is of course relative to results from similar cores using similar transformations. I have found that using the algorithm for first differences of the inverse-hypersine transformation works quite well and that $RATIO_{DN}$ values of 20 to 50 can be considered large. However, the user needs to gain experience with their own choice of transformation.

Another useful statistic is the ratio of the least cost path to the next to least cost path, $COST(path2)$. Call this $RATIO_2$:

$$RATIO_2 = 100[COST(path2) - COST(path1)]/COST(path2)$$

A relatively large value of $RATIO_2$ gives an indication that competing paths are poor in comparison to the best path. A small value suggests that alternative cross-dating options exist that are nearly as good as the best choice. Of course, one must remember that the dynamic programming algorithm does not keep track of all possible paths' values, since suboptimal paths are dropped as soon as they are detected. However, $RATIO_2$ and $RATIO_{DN}$ in combination give a good indication of whether the algorithm's advice should be taken seriously.

Limiting double and missing rings

The cross-dating algorithm is basically a vacuous data editor that strives to make the data appear to be perfectly cross-dated. If left uncontrolled, it may inject an exorbitant number of missing and double rings. The user of the algorithm should have some feeling for how many missing or double rings can reasonably occur over an interval, and it is desirable to allow the user to increase the cost of inserting missing or double rings to limit their use. A simple way of accomplishing this weighting is to add the cost of accepting the ring to the cost of inserting a missing or double ring.

The costs defined previously can then be redefined as follows:

$$\begin{aligned} a_u[y, t] &= a_u[y, t] + WTM \, a_m[y, t] \\ a_d[y, t] &= a_d[y, t] + WTD \, a_m[y, t] \end{aligned}$$

where WTM and WTD are penalty weights assigned by the user for allowing missing and double rings, respectively. These weights can then be increased until the algorithm uses missings or doubles in a very judicious manner.

Dating floating chronologies

Long tree-ring chronologies are often built up with wood from many sources; support timbers in old buildings may be cored, or preserved wood might be obtained from peat bogs or lakes. When material from sources other than living trees is used, floating chronologies may be formed that must be cross-dated with another existing chronology that has already been precisely dated. Morgan et al. (1987) discuss such a floating chronology in the British Isles that spans approximately the period 4500–3900 BC.

The dynamic programming algorithm discussed previously could also be used to assist in matching floating chronologies to a known chronology. The modification required is essentially putting the entire algorithm inside a loop. The loop would begin by assuming that the most recent year of the floating chronology matches the most recent year of the known chronology. The algorithm would then proceed to find the best fit or cheapest path for matching the floating and known chronologies from this starting point. The loop would then be incremented and the best fit found, assuming that the second ring of the known chronology is the correct starting point. This looping process would continue until all reasonable starting points on the known chronology have been evaluated.

The result of this looping process would be a series of costs associated with the best paths found for each starting point. One would expect to find the costs noticeably diminished in the vicinity of the correct starting year. The correct starting year should be associated with the minimum cost. This process would undoubtedly be time-consuming, but should locate the correct temporal vicinity where the floating chronology fits.

Summary

A dynamic programming algorithm has been described to aid in cross-dating tree-ring series. The algorithm will not replace the need for human intervention in the process, but may make cross-dating large data sets less time-consuming. The algorithm efficiently considers all possible combinations of accepting a ring as being correctly dated, inserting a missing ring, or adding two rings together to form a double ring. The cost involved with each decision is determined by computing the squared distance of the subject core from an assumed correctly dated chronology, where the subject core contribution is conditional on the particular cross-dating sequence being considered at that stage of the algorithm's progression. If the algorithm's declared best path includes an unreasonable amount of missing and double rings, the user can increase the cost of using missings or doubles and rerun the problem.

The boundaries depicted in Fig. 3 limit the number of sequential missings or doubles and determine the number

of cross-dating paths that the algorithm must consider. However, these boundaries do not directly limit the total number of missing or double rings on the best path; the user can only limit these by increasing their cost. Therefore, the algorithm requires subjective input from the user and is unable to function properly without a knowledgeable operator. In fact, the algorithm cannot proceed at all until provided with a reference set of correctly dated material.

This particular algorithm for cross dating is quite different from previously used methods that evaluate cross correlation between the subject core and a chronology; however, the two methods should be complementary. The dynamic programming approach described here relies heavily on the assumed cost functions. Although many cost functions could be used, minimizing a squared error loss function was the approach taken here. This does not necessarily maximize the overall correlation of the subject core with the reference chronology, but it is a reasonable criterion. The algorithm

and the cost function could be changed, but this research has demonstrated the potential of dynamic programming as a cross-dating tool.

- BAILLIE, M.G.L., and PILCHER, J.R. 1973. A simple crossdating program for tree-ring research. *Tree-Ring Bull.* **33**: 7-14.
- DREYFUS, S.E., and LAW, A.M. 1977. The art and theory of dynamic programming. Academic Press, New York.
- FRITTS, H.C. 1976. Tree rings and climate. Academic Press, London.
- HOLMES, R.L. 1983. Computer assisted quality control in tree-ring dating and measurement. *Tree-Ring Bull.* **43**: 69-78.
- MORGAN, R.A., LITTON, C.D., and SALISBURY, C.R. 1987. Trackways and treetrunks—dating Neolithic oaks in the British Isles. *Tree-Ring Bull.* **47**: 61-70.
- MUNRO, M.A.R. 1984. An improved algorithm for crossdating tree-ring series. *Tree-Ring Bull.* **44**: 17-28.
- YAMAGUCHI, D.K. 1986. Interpretation of cross correlation between tree ring series. *Tree-Ring Bull.* **46**: 47-54.